



ATM TRANSACTION SYSTEM

PROJECT BY
EFE KEMAL YILMAZ

AYKUT CINCIK

FATİH FURKAN ŞAHİN

STATEMENT OF WORK:

In this project, our group set the primary goal of solving the problem that banks are currently facing due to the rising number of customers. Our project aims to solve the problems that this rise causes, for instance, redundant operations that customers do regularly, increased operational cost resulting from the increase in the number of customers, and a quality drop of the service providing the functionality.

To achieve our goal, we propose the 'ATM Transaction System'. The ATM Transaction System is a distributed modular system that provides basic transactions such as money withdrawal/deposit and balance check through communicating with the Bank's Central System.

These transactions are implemented with the main goal of providing safety while being robust. The safety and robustness of the system are important for both banks and customers. Banks want their systems to operate under every condition while being safe because maintaining them all the time is not possible. Customers want safety and accessibility because they have their money in the bank. If the bank cannot provide safety, someone could use the customer's account without their permission. Also, if customers cannot access the system when they need to, it could result in distrust in the bank. Ultimately, these given factors could lead to customer dissatisfaction, thus doing harm to the bank.

This system will satisfy both banks and customers by providing a system that customers can use to do their monetary transactions by improving the overall quality of the service, thus relieving the banks by reducing operational costs and redundant operations.

System is acceptable for given items:

- Authentication must be done in 10 seconds.
- Implementation must be on Java.
- System must execute every scenario without runtime errors.

Delivery dates for this project are 14/12/2025. The deliverables for this project include Vision Statement, Requirements List, Use Case Definitions, Use Case Diagrams, Textual Analysis, UML Class Diagram, Implementation and Test Cases.

REQUIREMENTS LIST:

FUNCTIONAL REQUIREMENTS:

1. The Customer should be able to login to the System with a correct Card and PIN, and the System must automatically log out the user when the transaction is completed.
2. The Customer should be able to perform monetary transactions specifically: depositing money, withdrawing money, and checking the account balance.
3. The System must display a clear error message and cancel the operation if a transaction fails due to insufficient funds or system errors.
4. The System must keep the log to trace transactions.

NON-FUNCTIONAL REQUIREMENTS:

1. Customer Authentication must be done in 10 seconds.
2. Monetary Transactions must take less than 30 seconds.
3. Recording transactions must take less than 30 seconds.
4. Implementation must be on Java.
5. System must experience no runtime errors for valid inputs.

USE CASE DEFINITIONS:

1.

Use Case Name : CustomerAuthenticate

Participating: Initiated by **Customer**

Actors Communicates with **Bank Central System**

Flow of Events :

1. The Customer inserts the ATM card into the card reader.
 2. The System retrieves the credentials from the card.
 3. The System prompts the Customer to enter their PIN to log into account.
 4. The Customer enters the PIN using the keypad.
 5. The System validates the PIN with the Bank Central System.
 6. The System authenticates the user and authorizes access.
 7. The System displays the main transaction menu.
 8. Task is completed.
-

Alternate Steps :

- 2.1 The System could not retrieve the credentials from the card.
 - 2.2 Go to step 1.

 - 5.1. The Customer enters an incorrect PIN.
 - 5.2. The System displays an "Incorrect PIN" error message.
 - 5.3. The System checks retry attempts.
 - 5.4. Go to step "3."
-

Entry Conditions: Customer has inserted the card.

Exit Conditions: Customer presses "Cancel" button on suitable steps on transaction.

If the customer enters incorrect credentials three times, the system stores the card. (This is implemented as code termination.)

Task is completed.

Quality : The transaction steps must be completed without runtime errors.

Requirements Authentication and validation must be completed in less than 10 seconds.

2.

Use Case Name : MonetaryTransaction

Participating : Initiated by **Customer**

Actors Communicates with **Bank Central System**

Flow of Events :

1. The Customer selects a transaction between “Deposit, Withdraw, Check Balance”.
 2. If “Deposit” is selected, the System proceeds to execute it.
 3. The Customer puts money into the cash slot.
 4. The System counts the amount and communicates with the Bank Central System to update the balance.
 5. The System returns the card to the Customer.
 6. The System logs the transaction details (Type: Withdraw, Amount, Date/Time).
 7. Go to step “19.”.
 8. If “Withdraw” is selected, the System proceeds to execute it.
 9. The System requests an amount, and the Customer enters it.
 10. The System checks if the amount is within the account balance limit.
 11. The System communicates with the Bank to update the balance.
 12. The System returns the card to the Customer.
 13. The System dispenses the withdrawn money.
 14. The System logs the transaction details (Type: Withdraw, Amount, Date/Time).
 15. Go to step “19.”.
 16. If “Check Balance” is selected, the System proceeds to execute it.
 17. The System retrieves balance data from the Bank and displays it.
 18. Go to step “1.”.
 19. Task is completed.
-

Alternate Steps :

- 9.1. The Customer does not have enough cash in account.
 - 9.2. The System displays error message.
 - 9.3. Go to step “9.”.
-

Entry Condition : This use case can only be initiated when CustomerAuthenticate finished successfully then Customer selects to proceed **MonetaryTransaction** use case.

Exit Conditions : Customer presses “Cancel” button on suitable steps on transaction.
Task is completed.

Quality : The transaction steps must be completed without runtime errors.

Requirements Transactions must take less than 30 seconds.

3.

Use Case Name: *MonetaryTransactionLogs*

Participating: *Communicates with Bank Central System*

Actors

Flow of Events :

1. The System captures transaction details (Transaction Type, Amount, Date, Time, Card ID) immediately after the balance update.
2. The System formats these details into a log entry.
3. The System sends the log entry to the Bank Central System.
4. Task is completed.

Alternate Steps :

-

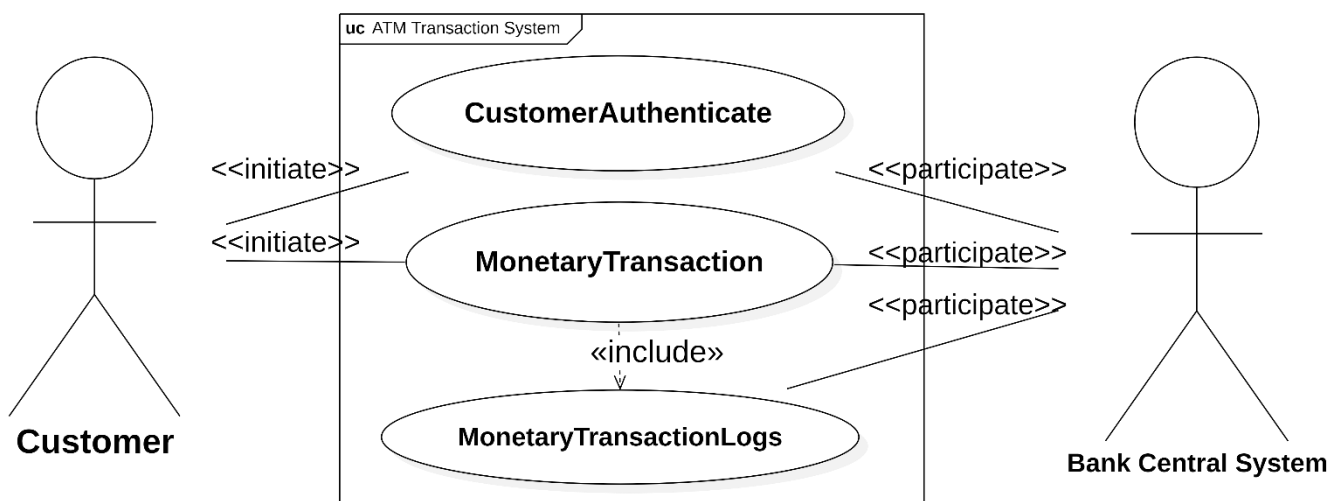
Entry Condition : This use case **included by** the **MonetaryTransaction**, it is initiated when Customer selects. "Deposit" or "Withdraw" proceed **MonetaryTransaction** use case and a transaction is completed.

Exit Conditions : Customer presses "Cancel" button on suitable steps on transaction.
Task is completed.

Quality : The transaction steps must be completed without runtime errors.

Requirements Transactions must take less than 30 seconds.

Use Case Diagram:



Textual Analysis:

NOUN ANALYSIS:

Noun / Noun Phrase	Role	Create a Class?	BCE Type	Reason
Customer	Actor	NO	-	It is an actor that uses the System.
Bank Central System	API of Bank	YES (to simulate)	Boundary (BankCentralSystem)	It is an interface that System communicates to perform some of the operations.
Account	Customer Account that holds attributes	YES	Entity (CustomerAccount)	It holds attributes for Customer.
PIN	Pin attribute	NO	-	It is an attribute in Account class.
Credentials	PIN and Card ID	NO	-	This project does not need Credentials because project already have both attributes.
Balance (Account Balance)	Available Money amount of customer	NO	-	It is an attribute in Account class.
ATM Card (Card)	Holds Card ID	YES (but NO)	Entity (Card)	It is only providing Card ID; in implementation our group are going to simulate it.
Card ID	An attribute of Card ID	NO	-	It is an attribute in Bank Central System.
Card Reader	Reads Card ID	YES (but NO)	Boundary (CardReader)	It only reads Card ID, in implementation this project going to simulate it.
Keypad	Provides keys to make inputs possible	YES (but NO)	Boundary (Keypad)	It provides buttons to give input; in implementation this project is going to simulate it.
Cash Slot	Location where money is put/taken	YES (but NO)	Boundary (CashSlot)	It provides an entry/exit point for cash; in implementation this project is going to simulate it.
Money (Cash)	Currency	NO	-	It is a real-life object.
Amount	Parameter	NO	-	It is a parameter to be passed in methods.
System	Generic name	NO	-	It is every class combined performing functionality of ATM Transaction System. The concept can be broken down into parts, so we can not say it is a class as a whole.

Transaction	Selected operation	NO	-	It is an input event from the Customer.
Transaction Menu	UI	YES	Boundary (ATMUI)	This project is going to implement this as CLI which is going to be projects UI.
Transaction Type	Attribute	NO	-	It is a parameter for logging methods.
Log File	Data	NO	-	It is a data that holds <i>Transaction Type, Amount, Date, Time, Card ID</i> .
Session	Loops	NO	-	Every iteration in UI main loop.
Error Message	A message	NO	-	A text that explains the error.
Date	Today's day and month	NO	-	Current day and month that will be passed as parameter.
Time	Current hour and minute	NO	-	Current hour and minute that will be passed as parameter.

Additional Classes	Role	BCE Type
ATMController	Responsible for the main logic of the system.	Controller
InputController (not implemented)	Handles the input logic of UI elements.	Controller
OutputController (not implemented)	Handles the output logic of ATM such as dispensing money etc.	Controller

VERB ANALYSIS:

Verb / Verb Phrase	Method?	Related Class	Method Name	Reason / Usage
inserts the ATM card	NO	-	-	It is an event that Customer does.
enters the PIN	NO	-	-	It is an event that Customer does.
selects a transaction	NO	-	-	It is an event that Customer does.
Deposit	YES	ATMController	deposit	Adding money to the account balance.
Withdraw	YES	ATMController	withdraw	Getting money then removing from the account balance.
Check Balance	YES	ATMController	checkBalance	Returning account balance.
puts money into the deposit slot	NO	-	-	It is an event that Customer does.
enters an amount	NO	-	-	It is an event that Customer does.
presses "Cancel" button	NO	-	-	It is an event that Customer does.
retrieves the credentials	YES (but no)	InputController	readCard	In this project it is simulated with a scanner that reads CLI line.
prompts the Customer	NO	-	-	This is in UI as print statement.
validates the PIN	YES	BankCentralSystem	validatePIN	System checks PIN against the Bank database (simulated with "CustomerAccount" variable in the code.)
authenticates the user	YES	ATMController	authCustomer	System gives permission to proceed.
authorizes access	NO	-	-	It is a return from authCustomer (true/false).
displays the main transaction menu	YES	ATMUI	showMainMenu	UI needs to show specific options.

displays error message	YES	ATMUI	displayErrorMenu	UI needs to show error text.
checks retry attempts	NO	-	-	Loop logic.
retains the card	YES (but no)	InputController	retainCard	Does not return the card.
terminates the session	NO	-	-	Exiting the loop.
proceeds to execute	NO	-	-	Pre-condition to enter the loop.
counts the amount	YES (but no)	OutputController	countMoney	Hardware must count the physical money.
communicates with Bank Central System	NO	-	-	Using methods of BankCentralSystem with the help of API.
updates the balance	YES	BankCentralSystem	updateBalance	System sends new balance to Bank database (simulated with "balance" variable in the CustomerAccount class.)
returns the card	YES (but no)	OutputController	ejectCard	Hardware action to give card back.
requests an amount	NO	-	-	This is in UI as print statement.
checks account balance limit	NO	-	-	Internal business logic inside the withdraw class.
dispenses the withdrawn money	YES (but no)	OutputController	dispenseCash	Hardware action to give money.
logs the transaction details	YES	ATMController	logTransaction	Main method to logging history.
captures transaction details	NO	-	-	Internal logic (gathering variables).
formats details	NO	-	-	Internal string manipulation logic.
sends the log entry	NO	-	-	Communication with Bank Central System.

UML Class Diagram:



Testing:

In our project, our group have written test cases to ATMControllerTEST.java (using JUnit) for ATMController.java . ATMController defines the logic of the system, ATMControllerTEST tests logical correctness. There are 10 tests which ATMController must execute logically correct to prove it's correctness. The names for implemented test cases are:

Test Case Name	What case "Tests"
testSetCurrentAccount_InvalidCardNumber	If provided card number is not recognized by the system, current account should be null.
testAuthCustomer_WrongPin	If provided pin is not related to current account, it should return "false".

testAuthCustomer_CorrectPin	If both credentials are related to the account, system should return "true".
testCheckBalance	Return amount must be the same with CustomerAccount class.
testDeposit	Deposit should change the account balance by the right amount.
testWithdraw_InsufficientFunds	Withdraw should not execute if the amount is not between the range.
testWithdraw_SufficientFunds	Withdraw should execute if the amount is between the range.
testLogOut	Logging out should make reset current customer.
testLogging	Functionality of logging (saving transactions)
testAuthCustomer_3TimesWrongLog	System should terminate itself if 3 consecutive wrong "log in".

The outputs for given test cases are:

```

TEST PASSED: Invalid card number scenario.
TEST PASSED: Wrong PIN check.
TEST PASSED: Correct PIN check.
TEST PASSED: Check Balance operation returned correct amount.
TEST PASSED: Deposit operation with amount 1000.00, Current Balance: 6000.00
TEST PASSED: Insufficient funds check with amount 6001.00, Current Balance: 6000.00
TEST PASSED: Withdraw operation with sufficient amount 3000.00, Current Balance: 3000.00
TEST PASSED: Logout operation.
Verified Log 1: [2025-12-10 15:26:42] WITHDRAW - Amount: 3000.0 - Card: 12345
Verified Log 1: [2025-12-10 15:26:42] DEPOSIT - Amount: 1000.0 - Card: 12345
=== 3 CONSECUTIVE WRONG PINS SCENARIO STARTING ===
1. Correct PIN wrong card attempted. Result: false
2. Wrong PIN correct card attempted. Result: false
3. Wrong PIN wrong card attempted. Result: false
TEST PASSED: Remaining attempts '0'
=== SCENARIO COMPLETED SUCCESSFULLY ===

```

EXPLANATION OF TESTING LOGIC:

Test Case Flow: "Authentication to the System"

Test Case Goal: to determine logic of logging into system works as expected.

1. User enters right credentials:

a.

```
Please enter Card Number (For simulation, press X to show logs.):
12345
Please enter PIN:
1234|
```

b. Login successful, Monetary Transaction Menu is available.

```
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)
```

2. User enters wrong credentials 3 times in a row:

```
Please enter Card Number (For simulation, press X to show logs.):
1234
Please enter PIN:
1234|
```

```
Card Number or PIN is invalid, 2 attempts are left.
```

```
Please enter Card Number (For simulation, press X to show logs.):
12345
Please enter PIN:
123|
```

```
Card Number or PIN is invalid, 1 attempts are left.
```

```
Please enter Card Number (For simulation, press X to show logs.):
123
Please enter PIN:
123|
```

```
Card Number or PIN is invalid, 0 attempts are left.
PS C:\Users\efeke\Desktop\principleYeniOdev\ProjectName> |
```

(terminates the program)

Correctness validation of the Code:

The test passed three possible cases: the wrong pin with the correct number, the right pin with the wrong number, and the right pin with the correct number. Only the correct pin with the correct number should proceed to the next step. Each possibility is tested, and the only correct pin with the correct number that confirms correctness proceeds to the next step.

Test Case Flow: "Making Monetary Transactions"

Test Case Goal: To determine if monetary transactions are working as intended.

1. User performs transaction:

a. User enters "1" (Choses Deposit):

```
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)

1|
```

b. User enters the amount "1000":

```
Please enter the amount you want to deposit (Press X to cancel):
1000|
```

c. The System indicates deposit is successful.

```
Deposit Successful!
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)

|
```

d. User enters "3" (Choses Check Balance):

```
Deposit Successful!
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)

3|
```

e. The System shows the balance:

```
Current balance: 6000.00 TRY
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)

|
```

- f. User enters "2" (Choses Withdraw):

```
Current balance: 6000.00 TRY
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)
```

2|

- g. User enters the amount "6001":

```
Please enter the amount you want to withdraw (Press X to cancel)
(Current Balance: 6000.00 TRY):
6001|
```

- h. The System warns User to enter an amount in balance range:

```
Please enter an amount in range of your balance (Current Balance: 6000.00 TRY):
```

|

- i. User enters the amount "3000":

```
Please enter the amount you want to withdraw (Press X to cancel)
(Current Balance: 6000.00 TRY):
3000|
```

- j. The System indicates withdraw is successful.

```
Withdraw Successful!
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)
```

|

- k. User enters "3" (Choses Check Balance):

```
Current balance: 3000.00 TRY
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)
```

|

- l. User enters "1" (Choses Deposit):

```
Current balance: 3000.00 TRY
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)
```

```
1|
```

- m. User enters "x" (Cancels the Transaction)

```
Please enter the amount you want to deposit (Press X to cancel):
```

```
x|
```

- n. The System redirects to menu section:

```
Deposit canceled.
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)
```

```
|
```

- o. User enters "4" (Terminate transactions, Log out)

```
Deposit canceled.
Please select the transaction:
1-Deposit
2-Withdraw
3-Check Balance
4-Cancel (Log out)
```

```
4|
```

- p. The System redirects to login section:

```
Please enter Card Number (For simulation, press X to show logs.):
```

```
|
```

- q. (IMPORTANT NOTE: this is for simulative purposes only, there is no such option in ATM, but we had to include it to show logging mechanism working properly.) To show previous transactions, User enters "X":

```
Please enter Card Number (For simulation, press X to show logs.):
```

```
x|
```

- r. The System shows logs:

```
--- TRANSACTION HISTORY (LOGS) ---  
[2025-12-04 23:24:17] DEPOSIT - Amount: 1000.0 - Card: 12345  
[2025-12-04 23:24:22] WITHDRAW - Amount: 3000.0 - Card: 12345  
-----  
  
Press ENTER to continue  
|
```

Correctness validation of the Code:

The test passed every possible interaction by chaining every one of them: Deposit, Withdrawal, Balance Check, cancelling a transaction and logging out. First, we made a deposit, which correctly updated the balance. Then, we attempted a withdrawal, which initially exceeded the balance, prompting the system to warn us that we had entered an amount within the range. Then, the System correctly updated the balance when the User entered an amount within the range. The User cancelled a transaction that had no change in the balance, demonstrating that the transaction was working correctly. The User printed the records to demonstrate the recording system (not included in the user interface; it's a database operation, but the System simulated it). The records were correct because they only recorded completed transactions that confirmed the correctness of code.
